

一、填空题

1. Unix、Linux 创建进程是用 fork()

EPOS 创建线程是用 task_create()

2. 系统结构 _____

简单结构: Unix 和 MS-DOS

分层结构: THE

微内核结构: Mach (用作 macOS 的内核), 微软 Windows NT/XP

虚拟机结构: VMware, Virtual Box, QEMU, Bochs, JVM

3. 术语解释: PCB 进程控制块 PTE 页表项

4. 内核中用 中断向量号 (Interrupt Vector Number) 区分中断, 用 系统调用号 实现系统调用

5. 内核从外部加载到内存并执行的程序是 boot-loader

6. 判断是 I/O port 还是 Memory-mapped I/O

显存 Memory-mapped I/O

键盘控制 I/O port

7. 设备保护 CPU 不被某进程长期占用的方法是 定时器 (timer)

8. EPOS 虚拟机

9. nice 静态、priority 动态 静态/动态优先级

10. 何时调度空闲线程 仅当没有其他可运行的线程时, 才能调度

11. 两个函数: 唤醒 wake_up 和睡眠 sleep_on

12. 判断以下代码会发生 死锁

```
sem_t s[5] = {1, 1, 1, 1, 1};
do {
    ...
    P(&s[i])
    P(&s[(i+1)%5])
    ...
    V
    V(&s[i])
} while(1)
```

13. 二元信号量实现计数型 (代码默写)

Implement counting semaphore using binary semaphore

```
typedef struct {
    int value;

    binary_semaphore bS1 = true, /*protect 'value'*/
                    bS2 = false; /*synchronization*/
} semaphore;

void P(semaphore *S)
{
    bP(&S->bS1);
    S->value--;
    if (S->value < 0) {
        bV(&S->bS1);
        bP(&S->bS2);
    } else
        bV(&S->bS1);
}

void V(semaphore *S)
{
    bP(&S->bS1);
    S->value++;
    if (S->value <= 0)
        bV(&S->bS2);
    bV(&S->bS1);
}
```

14.考察最坏适应

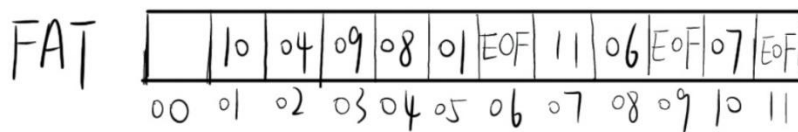
15.工作集是 局部性模型 的近似，问 $WS(t, \Delta)$ ，类似于下面的题目

9、工作集（Working-set）是对一个进程最近使用页面数量的近似。给定下列内存引用

0, 9, 0, 1, 8, 1, 8, 7, 8, 7, 1, 2, 8, 2, 7, 8, 2, 3, 8, 3

当 $t=12$, $\Delta=8$ 时, $Ws(t, \Delta)=$ 1、2、7、8。(t 从 1 开始)

16.FAT



起始块在 02 忘记要求什么了, 但是思路如下:

02 → 04 → 08 → 06 → EOF

17.求文件大小，类似于下面的题目

10、一个文件系统的 FCB（File Control Block）能存储 13 个磁盘块地址，其中前面的 10 个地址指向文件的前 10 个磁盘块，第 11 个地址指向一个一级索引块（single indirect block），第 12 个地址指向一个二级索引块（double indirect block）；第 13 个地址指向一个三级索引块（triple indirect block）。假设一个磁盘块有 4096 字节，每一个磁盘块可以存储 1024 个磁盘块地址。该文件系统能存储的最大的文件是 $(10+1024+1024*1024+1024*1024*1024)*4096$ (???) 字节。（只列出表达式即可）

二、大题

1.安全序列

2.1 (10 分) 一个系统有 5 个进程 P0-P4 和 4 种资源类型 (A, B, C, D)。T0 时刻系统状态如表所示:

进程	最大需求				已分配				可用			
	A	B	C	D	A	B	C	D	A	B	C	D
P0	0	0	1	2	0	0	1	0	1	5	2	0
P1	1	7	5	0	1	0	0	0				
P2	2	3	5	6	1	3	5	4				
P3	0	6	5	2	0	6	3	1				
P4	0	6	5	5	0	0	1	4				

- (1) 该状态是否安全? 如果是, 请写出一个安全序列 (safety sequence)。(5 分)
 (2) T0 时刻, P1 再请求 (0, 4, 2, 0), 系统能否分配给它? 为什么? (5 分)

2. 甘特图平均等待时间 (应该是三种: FCFS, SJF, RR)

四种算法, 画出甘特图, 并计算平均等待时间

P₁ 11

P₂ 28

P₃ 5

P₄ 8

P₅ 13

3. 页面缺页异常, 行号

页大小 2^8
 [128][64] } 自行替换题中数据
 i=27, j=4 } 有几帧忘记了
 掌握套路后问题不大

2.2 (32 分) 假设一个 32 位系统的页面大小为 4096 字节，给某个进程分配了 7 个帧：

序号	物理地址 (十六进制)
0	08000000
1	06000000
2	1C000000
3	0C000000
4	12000000
5	16000000
6	18000000

采用 LRU 页面置换算法，该进程按照如下两种方式运行：

```

int a[][] = new int[256][256];
for(int j = 0; j < 256; j++)
    for(int i = 0; i < 256; i++)
        a[i][j] = 0;
  
```

(A)

```

int a[][] = new int[256][256];
for(int i = 0; i < 256; i++)
    for(int j = 0; j < 256; j++)
        a[i][j] = 0;
  
```

(B)

假设该进程的地址空间大小也为 7 个页面，进程代码占用了页面 0 并已经映射到帧 0，且常驻物理内存。其余 6 页面用来保存数组 a 的数据，但尚未映射到物理内存。临时变量 i 和 j 保存在寄存器中，一个“int”占用 4 个字节。

- (1) 计算 A 和 B 各自从 $i=0, j=0$ 运行到 $i=28, j=7$ 时发生了多少次缺页异常。(8 分)
- (2) 对于 A 和 B，分别计算 $i=28, j=7$ 时帧 1-6 所存储的数组 a 的行号。(12 分)